

D-Voting: An End-to-End Verifiable Electronic Voting System Using Blind Signatures, Homomorphic Encryption and a Permissioned Audit Ledger

Pranav Kumar Singh¹, Dr. Kavi Priya D²

¹Department of Computing Technologies, Faculty of Engineering and Technology,

SRM Institute of Science and Technology (SRMIST), Kattankulathur, Chengalpattu, Tamil Nadu, India

M.Tech – Computer Science and Engineering (Full Stack), Roll No. RA2512055010005

²Department of Computing Technologies, Faculty of Engineering and Technology,

SRM Institute of Science and Technology (SRMIST), Kattankulathur, Chengalpattu, Tamil Nadu, India

Corresponding author: ps9567@srmist.edu.in

Abstract

Electronic voting must combine eligibility, ballot secrecy, correct recording and trustworthy counting while remaining understandable to voters. This paper presents D-Voting, a TypeScript prototype that composes end-to-end verifiability with a permissioned audit ledger. A registration authority issues one anonymous voting credential per electoral-roll entry using an RSA blind-signature protocol. The browser encrypts selections using exponential ElGamal and produces zero-knowledge validity proofs. A ballot-box service verifies the credential and proofs, while independent Ed25519 validators commit accepted records to a hash-chained ledger with Merkle roots. After closure, trustees jointly decrypt only the homomorphic aggregate and publish proofs for independent recounting. A Benaloh cast-or-audit challenge provides a probabilistic check against a client that encrypts a different choice from the one displayed. An observed local test run passed 516 of 516 tests. The system is presented as a reproducible research prototype rather than a production replacement for public elections: electoral-roll governance, endpoint compromise, timing correlation, coercion, trustee collusion and operational key management remain important limitations.

Keywords: electronic voting; end-to-end verifiability; blind signatures; ElGamal; zero-knowledge proofs; threshold decryption; public auditability.

1 Introduction

Internet voting can reduce geographic and administrative friction, but moving the ballot to a browser changes the trust problem. A central server can authenticate a voter, store a choice and return a count, yet the voter has little evidence that the client encrypted the intended choice, that the record was not changed, or that the result was computed from accepted ballots. A blockchain can make records tamper-evident, but it cannot by itself establish voter eligibility, ballot intent or coercion resistance [1].

D-Voting addresses this challenge as a composition of independently checkable controls. The registration service knows eligibility but signs a hidden credential. The browser retains the choice and encryption randomness while creating a proof of ballot validity. The ballot box validates without receiving civil identity. Validators replicate and attest to an append-only public record. Trustees verify that record and jointly release a tally from encrypted totals. The design follows the open-audit direction of Helios [2] while adding a permissioned ledger and threshold ceremony.

The contributions are: (i) a complete runnable architecture connecting anonymous registration, encrypted casting, public recording and threshold tallying; (ii) browser-side validity proofs and cast-or-audit checking; (iii) a permissioned ledger with independent validator replicas and Merkle inclusion proofs; (iv) a reproducible test and demonstration path; and (v) an explicit analysis of residual risks.

2 Background and requirements

2.1 End-to-end verifiability

End-to-end verifiable voting separates cast-as-intended, recorded-as-cast and tallied-as-recorded checks. No single server should be trusted for all three. Helios demonstrates public audit evidence for suitable low-coercion elections [2]. D-Voting applies the same separation to a multi-service prototype.

2.2 Requirements

Requirement	Design response	Evidence
Eligibility once	Electoral roll plus blind-signature issuance	Registration tests
Ballot secrecy	Browser-side ElGamal	Ciphertext boundaries
Ballot validity	Zero-knowledge proofs	Proof tests
Recorded-as-cast	Hash chain and Merkle path	Browser verifier
Tally integrity	Homomorphic aggregate and threshold proofs	Public recount
Lifecycle integrity	Opening and closing entries sealed to chain	Lifecycle tests

2.3 Threat model

The model includes malformed or replayed submissions, a dishonest ballot-box operator, a malicious frontend, colluding trustees, timing observers and a coercer. Fewer than the threshold number of trustees are assumed to collude, and validator authorities are expected to operate independent keys. These are assumptions, not properties guaranteed by code alone.

3 System architecture

The TypeScript monorepo contains `packages/crypto`, `packages/ledger` and services for registration, ballot-box, validators and trustees. The cryptographic package uses WebCrypto and native `BigInt`. The ledger package implements canonical encoding, block validation, Merkle proofs and replication. Hono-based services expose HTTP APIs, while browser pages support voting, verification, results, chain inspection and administration.

Party	Knows	Does not receive	Responsibility
Registration authority	Roll and issuance state	Unblinded credential or vote	Authenticate and sign
Voter browser	Choice and randomness	Other voters' data	Blind, encrypt and prove
Ballot box	Credential and ciphertext	Civil identity and plaintext	Validate and publish
Validator	Blocks and signatures	Plaintext vote	Re-verify and attest
Trustee	One key share	Civil identity	Verify and decrypt share

The election definition contains candidates, selection limits, schedule, issuer key, election public key, trustee roster, validator set and roll commitment. Opening seals this configuration in the first block; closing is also a chain event. The ledger is an audit layer, while secrecy and validity originate in cryptography.

4 Cryptographic protocol

4.1 Anonymous registration

The client creates a random credential message and blinds an RSA-PSS encoding before sending it to the registration authority. After confirming the roll entry and one-credential invariant, the authority signs the blinded value. The client unblinds and verifies the signature. The authority sees eligibility but not the final credential message. The implementation follows RFC 9474 [3].

4.2 Exponential ElGamal

For group public key ($y=g^x$), a ballot component is $((A,B)=(g^r,y^rg^m))$. Ciphertexts multiply componentwise, so exponents add. Candidate totals can therefore be computed without decrypting individual ballots, and the final discrete logarithm is taken over a bounded tally range. Parameters follow the MODP family described in RFC 3526 [4].

4.3 Validity, cast-or-audit and tally proofs

Disjunctive Chaum–Pedersen proofs show that each ciphertext encrypts an allowed value without revealing which one. Selection-limit checks prevent inflated ballots. A Benaloh cast-or-audit challenge lets the voter spoil and inspect a prepared ballot. After closure, Pedersen distributed key generation prevents a single dealer from assembling the election secret; trustees verify the board and submit proved partial decryptions of the aggregate.

5 Ledger and public verification

The ledger uses canonical encoding, hash-linked blocks, Merkle roots and Ed25519 signatures. Each validator maintains its own replica and checks proposed blocks, including ballot proofs and state transitions. A quorum is required to seal a block. A tracking code identifies a public record; the browser verifier recomputes hashes, checks signatures and walks the Merkle path. Results are published only after closure and threshold evidence.

Artefact	Check	Claim
Opening block	Compare definition and keys	Rules governing the poll
Block header	Recompute hashes	Record not altered
Signatures	Verify quorum	Authorities attested
Merkle path	Recompute root	Ballot included
Tally proofs	Aggregate and verify shares	Result follows from board

6 Implementation and evaluation

The registration authority exposes issuer, registration, credential and roll routes. The ballot box exposes election, cast, audit, bulletin, ceremony and lifecycle routes. Validators run as separate processes, and trustees have independent consoles. The demonstration exercises registration, encryption, audit, ledger sealing, re-voting, verification, tallying and tamper detection.

Tests cover blind RSA, group operations, ElGamal homomorphism, zero-knowledge proofs, threshold operations, distributed key generation, Benaloh auditing, Merkle trees, canonical encoding, storage, consensus, registration, lifecycle, ballot validation, tally publication and trustee refusal cases. The observed full run passed 516 of 516 tests. The repository reports illustrative timings of 485 ms for ballot creation, 730 ms for verification, 98 ms for partial decryption and less than 1 ms to combine ten ciphertexts; these should be independently reproduced before being treated as benchmarks.

7 Security analysis and limitations

The protocol provides meaningful separation of knowledge and public evidence, but its security is conditional. An invented voter on the roll can receive a valid credential; freezing and committing the roll makes later additions visible but cannot prove the original roll was honest. Three colluding trustees in a three-of-five configuration can decrypt individual ballots. A bearer-token administrator lacks a built-in two-person rule. Registration and voting times can still be correlated.

A malicious application can display one selection and encrypt another; validity proofs still succeed because they prove well-formedness, not human intent. Cast-or-audit makes this probabilistically detectable but needs an independent audit environment. Re-voting mitigates coercion but may reveal that a replacement occurred. Small contests need minimum tally rules because a single aggregate can disclose one choice. Production deployment also requires TLS, mutual authentication, KMS/HSM custody, identity proofing, durable audit storage and independently hosted verification.

8 Discussion and future work

D-Voting demonstrates how anonymous credentials, client encryption, proof-carrying ballots, independent validators and threshold trustees can compose into a public-audit workflow. The central design conclusion is that the blockchain is an audit layer, not the source of ballot secrecy. Future work should add independent roll governance, named administrators with dual control, secure key ceremonies, signed reproducible frontend builds, standalone verification, durable ceremony state, external chain-head mirroring and accessibility testing. Ranked-choice and stronger coercion resistance require different protocol constructions.

9 Conclusion

This paper presented D-Voting, a runnable prototype for end-to-end verifiable electronic voting. It combines blind-signature credentials, browser-side ElGamal encryption, zero-knowledge ballot proofs, a permissioned audit ledger, Merkle evidence and threshold decryption of homomorphic totals. The implementation and tests provide an inspectable research artefact. It should be presented as a prototype for controlled research and teaching settings, not as a ready-to-deploy public-election platform.

References

- [1] S. Park, M. Specter, N. Narula and R. L. Rivest, “Going from bad to worse: from Internet voting to blockchain voting,” *Journal of Cybersecurity*, 7(1), 2021.
- [2] B. Adida, “Helios: Web-based Open-Audit Voting,” *USENIX Security Symposium*, 2008.
- [3] F. Denis, K. Jacobs and C. A. Wood, “RSA Blind Signatures,” RFC 9474, 2023.
- [4] T. Kivinen and M. Kojo, “More Modular Exponential (MODP) Diffie-Hellman groups,” RFC 3526, 2003.
- [5] B. Laurie, A. Langley and E. Kasper, “Certificate Transparency,” RFC 6962, 2013.
- [6] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm,” RFC 8032, 2017.
- [7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” 2023.

Submission checklist

- Replace author, affiliation, email and corresponding-author placeholders.
- Confirm authorship with the supervisor and include only substantial contributors.
- Repeat `npm test` and record the exact commit or release version.
- Adapt title, page limit, template and citation style to the chosen venue.
- Disclose AI assistance according to the target venue policy.